HOW-TO GUIDE | EDI-HOWTO-03 | Version 1.0

How to Test an EDI Pipeline Using pytest

A practical guide to creating a pytest module for the edi-integration-pipeline with one positive, one negative, and one API test, then running it and reading the results.

Applies to	Transactions	Task type	Estimated time
edi-integration-pipeline v1	X12 850, 997, and 810	Development	15–20 minutes

Introduction

The `edi-integration-pipeline` parses and validates inbound X12 850 purchase orders, translates accepted orders to JSON, generates X12 997 acknowledgments, and creates outbound X12 810 invoices from invoice data.

This guide shows how to create `tests/test_edi_howto.py`, a three-test tutorial module based on the completed repository test suite. You will create one test of each type, then execute them with pytest:

- **Positive test:** a valid 850 produces an accepted 997.
- **Negative test:** an invalid 850 produces a rejected 997.
- **API test:** a valid 850 posted to `/edi/850` returns an accepted response.

The repository's completed `tests/test_edi.py` contains 25 tests built with the same pattern. Do not overwrite it; this guide uses a separate file.

Scope

IN SCOPE

- Creating tests/test_edi_howto.py
- Adding the imports and shared setup it requires
- Writing one positive, one negative, and one API test
- Running all three tutorial tests or one specific test
- Interpreting PASSED, FAILED, and ERROR results

NOT COVERED

- Creating the Python environment
- Installing project dependencies
- Implementing pipeline functions or validation rules
- The full 25-test repository suite
- Full X12 implementation-guide compliance
- Testing a deployed API over a network
- CI execution with GitHub Actions
- Production or trading-partner data

Before You Start

This guide assumes that:

- The `edi-integration-pipeline` repository is already cloned.
- The Python virtual environment is active.
- The dependencies in `requirements.txt` are installed.
- You are working from the repository root.
- The `src/`, `input/`, and `tests/` directories are available.

© 2026 alowinarp · CC BY 4.0

Example

Every step uses two sample files from the pipeline's `input/` folder: `valid_850.txt`, a clean purchase order, and `invalid_850.txt`, which contains three known errors.

pytest discovers test functions by name. A Python file named `test_*.py` is eligible for collection, and functions whose names begin with `test_` are treated as tests.

Each test follows the Arrange–Act–Assert pattern:

1. **Arrange:** Prepare the input.
2. **Act:** Call the code being tested.
3. **Assert:** State the result that must be true.

An *assertion* is a statement that must be true for the test to pass. In Python, `assert` checks an expression: if it is true, the test continues; if it is false, pytest stops the test and reports it as `FAILED`. For example, `assert "AK5*A" in acknowledgment` means "the 997 must contain an accepted AK5 segment."

When an assertion fails, pytest displays the failing line and the value it actually received.

Steps

Follow these steps to build `tests/test_edi_howto.py`, then execute it.

1 Create the test module

In the repository's `tests/` directory, create:

```
tests/test_edi_howto.py
```

The expected project structure is:

```
edi-integration-pipeline/
├── input/
├── src/
├── tests/
│   ├── test_edi.py
│   └── test_edi_howto.py
├── requirements.txt
└── README.md
```

pytest will discover the file because its name begins with `test_`.

> **Reference copy**
> A completed copy of this file is in the playbook repository at howto/samples/test_edi_howto.py. If you get stuck, compare your file against it. To run the completed copy, place it in the pipeline's `tests/` folder.

2 Add the import-path configuration

The application modules are under `src/`. Add the following code at the beginning of `test_edi_howto.py` so Python can import them:

```
import os
import sys


SRC_FOLDER = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "..",
    "src",
)
sys.path.insert(0, SRC_FOLDER)
```

`__file__` identifies the location of `test_edi_howto.py`. The path moves up from `tests/` to the project root and then enters `src/`. This allows imports such as `from edi_parser import parse_edi` to work when pytest runs from the project root.

© 2026 alowinarp · CC BY 4.0

3 Import the code under test

Add these lines **below** `sys.path.insert(0, SRC_FOLDER)`. They must come after it, because Python can only find these modules once `src/` is on its search path.

```
from edi_997 import generate_997
from edi_parser import parse_edi
from fastapi.testclient import TestClient
from main import app
from validation.validate_850 import validate_850
```

Import	Purpose
`parse_edi`	Parse X12 text into segments and delimiters
`validate_850`	Validate selected 850 transaction rules
`generate_997`	Generate an accepted or rejected 997 acknowledgment
`app`	The FastAPI application under test
`TestClient`	Send in-process requests to FastAPI routes

4 Add the shared test setup

Create the FastAPI test client and a helper that loads sample EDI files from `input/`:

```
client = TestClient(app)

INPUT_FOLDER = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "..",
    "input",
)


def read_sample_850(file_name: str) -> str:
    path = os.path.join(INPUT_FOLDER, file_name)

    with open(path, "r") as open_file:
        return open_file.read()
```

`TestClient` calls the FastAPI application in the same Python process, without starting Uvicorn. The input path is derived from the location of `test_edi_howto.py`, so the helper does not depend on the terminal's current directory.

5 Add a positive test

A positive test confirms that valid input is accepted. This test validates a clean 850 and checks that the 997 accepts it:

```
def test_generate_997_accepts_a_valid_850() -> None:
    parsed = parse_edi(read_sample_850("valid_850.txt"))
    errors = validate_850(parsed.segments)

    acknowledgment = generate_997(
        parsed.segments,
        errors,
        parsed.delimiters,
    )

    assert "ST*997*" in acknowledgment
    assert "AK1*PO*101" in acknowledgment
    assert "AK2*850*0001" in acknowledgment
    assert "AK5*A" in acknowledgment
    assert "AK9*A*1*1*1" in acknowledgment
```

A pass means the valid sample produced the expected accepted transaction (`AK5*A`) and functional-group (`AK9*A`) acknowledgment values.

© 2026 alowinarp · CC BY 4.0

How this test is structured

Every test function in this guide is built the same way. If you are new to Python, use this breakdown to read the code above line by line:

Part	In the code	What it does
Definition	`def`	Starts a function: a named block of code that pytest can run.
Name	`test_generate_997_accepts_a_valid_850`	The `test_` prefix is how pytest finds the function. The rest describes what is being tested.
Inputs	`()`	Empty parentheses mean the function takes no inputs.
Return type	`-> None`	Documents that the function returns no value. Because there is no `return` statement, Python returns `None`.
Body	`:` and indentation	The colon ends the first line. Every indented line below it belongs to the function.
Arrange	`parsed = parse_edi(...)` `errors = validate_850(...)`	Reads the sample 850 and validates it. `=` stores each result in a variable for later lines.
Act	`acknowledgment = generate_997(...)`	Calls the code being tested and stores the 997 text it returns.
Assert	`assert "AK5*A" in acknowledgment`	`in` checks that the 997 text contains the segment. If any `assert` is false, the test fails.

6 Add a negative test

A negative test confirms that invalid input is rejected. The sample `invalid_850.txt` deliberately contains a missing BEG03 purchase-order number, a missing BEG05 purchase-order date, and a non-numeric PO1 quantity.

The structure is the same as the positive test. Only the input file and the expected values change: `AK5*R*5` means the transaction set was rejected with code 5 (one or more segments in error). `AK9*R*1*1*0` means the functional group was rejected; one transaction set was included, one was received, and zero were accepted.

```
def test_generate_997_rejects_an_invalid_850() -> None:
    parsed = parse_edi(read_sample_850("invalid_850.txt"))
    errors = validate_850(parsed.segments)

    acknowledgment = generate_997(
        parsed.segments,
        errors,
        parsed.delimiters,
    )

    assert "AK5*R*5" in acknowledgment
    assert "AK9*R*1*1*0" in acknowledgment
```

A pass means the invalid sample produced the expected rejected 997. It does not mean the input was processed as a valid purchase order.

© 2026 alowinarp · CC BY 4.0

7 Add an API test

An API test sends a request to a FastAPI route through `TestClient` and checks the status code and response body. Arrange reads the sample file, Act posts it to `/edi/850` with `client.post(...)`, and Assert checks the response:

```
def test_post_850_endpoint_accepts_a_valid_850() -> None:
    edi_text = read_sample_850("valid_850.txt")

    response = client.post(
        "/edi/850",
        content=edi_text,
        headers={"Content-Type": "text/plain"},
    )

    assert response.status_code == 200

    body = response.json()
    assert body["validation_status"] == "accepted"
    assert (
        body["purchase_order"]["purchase_order_number"]
        == "PO10001"
    )
    assert "AK5*A" in body["acknowledgment_997"]
```

This is an application-level test. It does not test a deployed server, network connectivity, TLS, authentication, or AS2 transport.

> **Important**
>
> A successful request calls the application's save functions and may overwrite these sample output files:
>
> - `output/997/PO10001_997.txt`
> - `output/850_json/PO10001.json`

8 Run the tutorial module

Activate the virtual environment (if needed)

If your prompt does not start with `(.venv)`, activate the environment first. From the repository root (`edi-integration-pipeline/`), run the command for your terminal:

Terminal	Command
Windows PowerShell	`.venv\Scripts\Activate.ps1`
Windows Command Prompt	`.venv\Scripts\activate.bat`
Git Bash on Windows	`source .venv/Scripts/activate`
macOS / Linux	`source .venv/bin/activate`

It worked if your prompt starts with `(.venv)`. To turn it off later, run `deactivate`.

Run the module

Save `test_edi_howto.py`. From the project root, run:

```
python -m pytest -v tests/test_edi_howto.py
```

`-v` displays the name and outcome of each collected test.

> **Expected result**
>
> pytest collects the three tests in `tests/test_edi_howto.py`, and the summary reports `3 passed`. The elapsed time varies by computer.
>
> A `StarletteDeprecationWarning` may also appear, and the summary then reads `3 passed, 1 warning`. The warning comes from a third-party library, not from your tests, and can be ignored.
>
> Include the file path. Running `python -m pytest -v` alone also collects the 25 tests in the repository's `tests/test_edi.py`.

© 2026 alowinarp · CC BY 4.0

9 Run one test at a time

To run a single test, add the file path and the function name. Run each of the three tests in turn:

```
python -m pytest -v tests/test_edi_howto.py::test_generate_997_accepts_a_valid_850
python -m pytest -v tests/test_edi_howto.py::test_generate_997_rejects_an_invalid_850
python -m pytest -v tests/test_edi_howto.py::test_post_850_endpoint_accepts_a_valid_850
```

> **Expected result**
>
> Each command reports one `PASSED` test:
>
> - **Positive:** the valid sample produced the accepted 997 values asserted by the test.
> - **Negative:** the invalid 850 was correctly rejected. A negative test passing is not an application failure; the rejection is the expected behavior.
> - **API:** an in-process request returned the asserted status code and JSON response. It does not prove that a deployed server or production integration is working.

10 Interpret failures and errors

Outcome	Meaning	Next action
`PASSED`	The observed result satisfied the assertions.	Continue with the remaining tests.
`FAILED`	The test ran, but at least one assertion was false.	Compare the observed value with the assertion, input fixture, and implementation.
`ERROR`	pytest could not collect or execute the test.	Check imports, paths, dependencies, syntax, and environment activation.
No tests collected	pytest did not discover the file or functions.	Confirm the file name begins with `test_` and each function name begins with `test_`.

> **Tip**
>
> Do not edit an assertion merely to make a test pass. Change it only when the intended behavior has deliberately changed.

Result

You have created a pytest module with a positive, a negative, and an API test, and you can now:

- Structure a test with Arrange–Act–Assert
- Run the tutorial module or an individual test
- Interpret passing tests, assertion failures, and setup errors

To add coverage, write another `test_` function with the same pattern and run the suite again.

? Need Help?

Found an issue or have a question?
Open an issue in the edi-integration-playbook repository on GitHub.

Document History

Version	Date	Author	Change
1.0	October 2026	alowinarp	Initial release

© 2026 alowinarp · CC BY 4.0

© 2026 alowinarp · CC BY 4.0